

DIRECTION NATIONALE DE L'ENSEIGNEMENT
SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE



Ecole Nationale d'Ingénieurs - Alderhamane Baba Touré

410, Av. Van Vollenhoven BP 242 – Tél : (223) 20 22 27 36 – Fax : (223) 20 21 50 38 / Bamako – MALI.

Département Informatique-Télécom Option Informatique

Thème : Communication entre client et
serveur

Master 1 GIT –S2

Objet	
Noms et Prénoms	Drissa Sidiki Traore Ibrahima Guindo
Dépôt : 20/12/2024	Groupe : N°

NOTE	APPRECIATION
..... /20	

ANNEE : 2023-2024

Table des matières

Introduction :	3
Exercices :	3
Exercice 1 : Communication entre un client et un serveur	3
Objectifs :	3
Code et Explications :	3
Captures d'écran des résultats	5
Exercice 2 : Communication entre plusieurs clients via un serveur multithreadé	5
Objectif :	5
Code et Explications :	5
Captures d'écran des résultats :	8
Conclusion :	8

Introduction :

Les technologies réseau jouent un rôle crucial dans la communication entre applications distribuées. Ce rapport présente deux exercices pratiques illustrant la communication client-serveur en utilisant des sockets et des threads pour gérer les échanges.

Les objectifs principaux sont de développer une compréhension des concepts de base des sockets, de la gestion des connexions réseau et de l'utilisation des threads pour permettre une communication simultanée entre plusieurs clients.

Exercices :

Exercice 1 : Communication entre un client et un serveur

Objectifs :

Créer une application où un **client** se connecte à un **serveur**, échange des messages, et termine la session via une commande spécifique (FIN).

Code et Explications :

Client :

Le client établit une connexion au serveur, envoie et reçoit des messages via des sockets.

Code utilisé :

```
1  # Définition d'un client réseau rudimentaire
2  # Ce client dialogue avec un serveur ad hoc
3
4  import socket
5  import sys
6
7  HOST = '127.0.0.1'
8  PORT = 50000
9
10 # 1) création du socket :
11 mySocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12
13 # 2) envoi d'une requête de connexion au serveur :
14 try:
15     mySocket.connect((HOST, PORT))
16 except socket.error:
17     print("La connexion a echoue.")
18     sys.exit()
19
20 print("Connexion etablie avec le serveur.")
21
22 # 3) Dialogue avec le serveur :
23 msgServeur = mySocket.recv(1024).decode("Utf8")
24
25 while True:
26     if msgServeur.upper() == "FIN" or msgServeur == "":
27         break
28     print("S>", msgServeur)
29     msgClient = input("C> ")
30     mySocket.send(msgClient.encode("Utf8"))
31     msgServeur = mySocket.recv(1024).decode("Utf8")
32
33 # 4) Fermeture de la connexion :
34 print("Connexion interrompue.")
35 mySocket.close()
36
```

Serveur :

Le serveur attend les connexions des clients et dialogue avec eux jusqu'à la réception du mot-clé FIN.

Code utilisé :

```
1 # Définition d'un serveur réseau rudimentaire
2 # Ce serveur attend la connexion d'un client
3
4 import socket
5 import sys
6
7 HOST = '127.0.0.1'
8 PORT = 50000
9 counter = 0 # compteur de connexions actives
10
11 # 1) création du socket :
12 mySocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
13
14 # 2) liaison du socket à une adresse précise :
15 try:
16     mySocket.bind((HOST, PORT))
17 except socket.error:
18     print("La liaison du socket à l'adresse choisie a échoué.")
19     sys.exit()
20
21 while True:
22     # 3) Attente de la requête de connexion d'un client :
23     print("Serveur prêt, en attente de requêtes ...")
24     mySocket.listen(2)
25
26     # 4) Etablissement de la connexion :
27     connexion, adresse = mySocket.accept()
28     counter += 1
29     print("Client connecté, adresse IP %s, port %s" % (adresse[0], adresse[1]))
30
31     # 5) Dialogue avec le client :
32     msgServeur = "Vous êtes connecté au serveur ENI_GIT. Envoyez vos messages."
33     connexion.send(msgServeur.encode("Utf8"))
34     msgClient = connexion.recv(1024).decode("Utf8")
35     while True:
36         print("C>", msgClient)
37         if msgClient.upper() == "FIN" or msgClient == "":
38             break
39         msgServeur = input("S> ")
40         connexion.send(msgServeur.encode("Utf8"))
41         msgClient = connexion.recv(1024).decode("Utf8")
42
43     # 6) Fermeture de la connexion :
44     connexion.send("fin".encode("Utf8"))
45     print("Connexion interrompue.")
46     connexion.close()
47
48     ch = input("<R> recommencer <T> terminer ? ")
49     if ch.upper() == 'T':
50         break
51
```

Captures d'écran des résultats

```
DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/server
/tp1
$ py server.py
Serveur pret, en attente de requetes ...
Client connecte, adresse IP 127.0.0.1, port 54085
C> Salam serveur comment tu vas ?
S> Halham doulilah ça va
.....
```

```
DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/server
/tp1
$ py client.py
Connexion etablie avec le serveur.
S> Vous etes connecte au serveur ENI_GIT. Envoyez vos messages.
C> Salam serveur comment tu vas ?
S> Halham doulilah ça va
C> .....
```

Exercice 2 : Communication entre plusieurs clients via un serveur multithreadé

Objectif :

Créer un serveur capable de gérer plusieurs clients simultanément, permettant une communication en temps réel entre eux grâce à des **threads**.

Code et Explications :

Serveur :

Le serveur utilise un thread distinct pour chaque client connecté. Les messages reçus sont diffusés à tous les autres clients connectés.

Code utilisé :

```

1 import socket
2 import sys
3 import threading
4
5 HOST = '127.0.0.1'
6 PORT = 46000
7 client_lock = threading.Lock()
8 conn_client = {} # Dictionnaire des connexions clients
9 running = True # Flag pour arrêter proprement le serveur
10
11 class ThreadClient(threading.Thread):
12     """Derivation d'un objet thread pour gerer la connexion avec un client"""
13     def __init__(self, conn):
14         threading.Thread.__init__(self)
15         self.connexion = conn
16
17     def run(self):
18         nom = self.name
19         print(f"Thread {nom} démarré pour gérer le client.")
20         while running:
21             try:
22                 msgClient = self.connexion.recv(1024).decode("Utf8")
23                 if not msgClient or msgClient.upper() == "FIN":
24                     break
25
26                 message = f"{nom}> {msgClient}"
27                 with client_lock:
28                     for cle in conn_client:
29                         if cle != nom:
30                             conn_client[cle].send(message.encode("Utf8"))
31             except Exception as e:
32                 print(f"Erreur avec le client {nom}: {e}")
33                 break
34
35         print(f"Fermeture de la connexion pour le client {nom}.")
36         self.connexion.close()
37         with client_lock:
38             del conn_client[nom]
39         print(f"Client {nom} déconnecté.")
40
41 def server_control():
42     global running
43     while running:
44         command = input("Tapez 'STOP' pour arrêter le serveur : ").strip()
45         if command.upper() == "STOP":
46             print("Arrêt du serveur...")
47             running = False
48             mySocket.close() # Ferme le socket principal
49
50 # Initialisation du serveur
51 mySocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
52 try:
53     mySocket.bind((HOST, PORT))
54     mySocket.listen(5)
55     print(f"Serveur démarré sur {HOST}:{PORT}, en attente de connexions...")
56 except socket.error as e:
57     print(f"Erreur : La liaison du socket a échoué - {e}")
58     sys.exit()
59
60 # Lancer un thread pour les commandes du serveur
61 thread_control = threading.Thread(target=server_control, daemon=True)
62 thread_control.start()
63
64 while running:
65     try:
66         connexion, adresse = mySocket.accept()
67         th = ThreadClient(connexion)
68         th.start()
69         it = th.name
70         with client_lock:
71             conn_client[it] = connexion
72         print(f"Client {it} connecté, adresse IP {adresse[0]}, port {adresse[1]}")
73         connexion.send("Vous êtes connecté. Envoyez vos messages.".encode("Utf8"))
74     except Exception as e:
75         if not running: # Arrêt via commande "STOP"
76             break
77         print(f"Erreur dans la boucle principale : {e}")
78
79 print("Serveur arrêté.")
80

```

Client :

Chaque client utilise deux threads pour gérer l'envoi et la réception de messages en parallèle.

Code utilisé :

```
1 import socket, sys, threading
2
3 host = '127.0.0.1'
4 port = 46000
5
6
7 class ThreadReception(threading.Thread):
8     """Objet thread gerant la reception des messages"""
9     def __init__(self, conn, stop_event):
10         threading.Thread.__init__(self)
11         self.connexion = conn # Référence du socket de connexion
12         self.stop_event = stop_event # Événement de signalisation pour arrêter
13
14     def run(self):
15         while not self.stop_event.is_set():
16             message_recu = self.connexion.recv(1024).decode("Utf8")
17             print("*" + message_recu + "*")
18             if not message_recu or message_recu.upper() == "FIN":
19                 break
20             print("Client arrete. Connexion interrompue.")
21             self.connexion.close()
22
23 class ThreadEmission(threading.Thread):
24     """Objet thread gerant l'emission des messages"""
25     def __init__(self, conn, stop_event):
26         threading.Thread.__init__(self)
27         self.connexion = conn # Référence du socket de connexion
28         self.stop_event = stop_event # Événement de signalisation pour arrêter
29
30     def run(self):
31         while not self.stop_event.is_set():
32             message_emis = input()
33             self.connexion.send(message_emis.encode("Utf8"))
34
35 # Programme principal - Établissement de la connexion :
36 connexion = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
37 try:
38     connexion.connect((host, port))
39 except socket.error:
40     print("La connexion a echoue.")
41     sys.exit()
42
43 print("Connexion etablie avec le serveur.")
44
45 # Créer un événement pour signaler L'arrêt des threads
46 stop_event = threading.Event()
47
48 # Dialogue avec Le serveur : on lance deux threads pour gérer L'émission et la réception des messages :
49 th_E = ThreadEmission(connexion, stop_event)
50 th_R = ThreadReception(connexion, stop_event)
51 th_E.start()
52 th_R.start()
53
54
```

Captures d'écran des résultats :

Ici, on a utilisé 2 clients et un serveur.

```
pwsh in tp2
tp2 py .\server.py
Serveur démarré sur 127.0.0.1:46000, en attente de connexions. Thread Thread-2 démarré pour gérer le client.
Tapez 'STOP' pour arrêter le serveur : Thread Thread-3 démarré pour gérer le client.
Client Thread-2 connecté, adresse IP 127.0.0.1, port 54183.
Thread Thread-3 démarré pour gérer le client.
Client Thread-3 connecté, adresse IP 127.0.0.1, port 54184.
```

```
pwsh in tp2
tp2 py .\client.py
Connexion établie avec le serveur.
*Vous êtes connecté. Envoyez vos messages.*
Bonjour à tous
*Thread-3> Quoi de neuf*
```

```
pwsh in tp2
tp2 py .\client.py
Connexion établie avec le serveur.
*Vous êtes connecté. Envoyez vos messages.*
*Thread-2> Bonjour à tous*
Quoi de neuf
```

Conclusion :

Ces exercices démontrent les bases de la communication réseau à l'aide de sockets en Python.

- L'exercice 1 met en évidence une **communication simple client-serveur**.
- L'exercice 2 illustre une **communication multiclients avec un serveur multithreadé**.

Les concepts appris ici sont essentiels pour développer des applications distribuées plus complexes comme des systèmes de chat ou des applications de collaboration en temps réel.